Trabalho de Conclusão de Curso de Engenharia da Computação

RADARTORRES:
PROTÓTIPO PARA DETECÇÃO, LOCALIZAÇÃO E SELEÇÃO AUTOMÁTICA DE TORRE COM CONTROLADOR HEURÍSTICO-NEUROEVOLUTIVO

RadarTorres:
a prototype for target detection, localization and automatic tower selection with a heuristic-neuroevolutionary controller

Gabriel Vasconcellos de Morais[1]
Matheus Emanoel Gomes Souza[2]
Anker Douglas Loss[3]

RESUMO

A construção de protótipos de sensoriamento com hardware acessível é um recurso didático relevante em Engenharia da Computação, por permitir explorar de forma prática conceitos de aquisição de dados, rastreamento e tomada de decisão automática. Este trabalho apresenta o RadarTorres, um protótipo que detecta um alvo em movimento, estima sua posição e decide, entre um conjunto de torres candidatas posicionadas ao redor de uma base central, qual delas deve responder a esse alvo – problema central que o trabalho busca resolver: como identificar, localizar e atribuir automaticamente a torre mais adequada a cada alvo, de forma auditável e sem depender de sensores ou atuadores de uso industrial. O sistema combina dois sensores ultrassônicos (que medem a distância pelo eco do ultrassom) com quatro câmeras de 720p, processa quadros de vídeo (*frames*) e classifica o alvo por sua cor, formato e tamanho. O protótipo converte as leituras de ângulo e distância em posição cartesiana, exibindo-as em tempo real em um radar circular, seleciona automaticamente a torre mais adequada para cada alvo e registra em auditoria toda ação executada. Essa seleção combina uma heurística determinística, escrita como regras explícitas de engenharia, com uma rede neural que aprende, por meio de um algoritmo genético próprio, a refinar essa escolha e o instante do acionamento. O treinamento dessa rede ocorre inteiramente antes da operação do sistema, em um ambiente simulado que executa gerações de episódios de teste até evoluir os parâmetros da rede; concluído o treinamento, esses parâmetros são incorporados de forma fixa à aplicação, de modo que, durante a execução real, o sistema apenas aplica o que foi aprendido previamente, sem ajustar seus parâmetros em tempo real. O processamento e a tomada de decisão são concentrados em uma aplicação desktop.

[1]Graduando(a) em Engenharia da Computação pela Faculdade do Centro Leste – UCL. E-*mail*: gabrielmorais@ucl.br

[2]Graduando(a) em Engenharia da Computação pela Faculdade do Centro Leste – UCL. E-*mail*: matheusemanoel@ucl.br

[3]Orientador(a). Professor especialista, bacharel em Ciência da Computação pela Universidade Federal do Espírito Santo (UFES). E-*mail*: anker@ucl.br

Palavras-chave: Detecção e rastreamento de alvos; Seleção automática de torre; Controlador heurístico-neuroevolutivo; Arduino; Protótipo.

ABSTRACT

This work presents RadarTorres, a prototype developed as a Final Course Project in Computer Engineering that detects a moving target, estimates its position, and decides, among a set of candidate towers placed around a central base, which one should respond to that target – the central problem this work addresses: how to automatically identify, localize, and assign the most suitable tower to each target, in an auditable way and without relying on industrial-grade sensors or actuators. The system combines two ultrasonic sensors (which measure distance by ultrasound echo) with four 720p cameras, processes video frames, and classifies the target by color, shape, and size. The prototype converts angle and distance readings into Cartesian position, displaying them in real time on a circular radar, automatically selects the most suitable tower for each target, and logs every executed action for auditing. This selection combines a deterministic heuristic, written as explicit engineering rules, with a neural network that learns, through a custom genetic algorithm, to refine that choice and the activation timing. Training of this network happens entirely before the system is put into operation, in a simulated environment that runs generations of test episodes until the network's parameters evolve; once training is complete, these parameters are embedded as fixed values into the application, so that, during actual execution, the system only applies what was previously learned, without adjusting its parameters in real time. Processing and decision-making are concentrated in a desktop application, with an architecture designed to allow future evolution without requiring additional hardware. Finally, the limitations inherent to the accessible, low-cost hardware approach adopted are discussed, as is the fact that validation of the prototype has so far been carried out entirely through software simulation, since the physical structure has not yet been assembled or tested.

Keywords: Target detection and tracking; Automatic tower selection; Heuristic-neuroevolutionary controller; Arduino; Prototype.

1 Introdução

Sistemas automáticos de detecção e resposta a alvos exigem, além de identificar um estímulo, rastrear com precisão sua posição e aplicar regras de decisão que evitem respostas indevidas – tarefas comumente resolvidas com sensores de radar, de custo elevado. Como alternativa a esse tipo de equipamento, este trabalho adota técnicas de visão computacional, aliadas a hardware acessível, como parte central da arquitetura do protótipo. Protótipos didáticos construídos dessa forma desempenham papel importante nesse processo, por permitirem que esses conceitos sejam explorados de forma prática e replicável em laboratório, reproduzindo os princípios de sensoriamento, rastreamento e decisão presentes em sistemas mais sofisticados.

O protótipo resultante combina sensores ultrassônicos e câmeras para detectar um alvo, estimar sua posição em um radar circular e decidir, entre um conjunto de torres candidatas dispostas ao redor de uma base central, qual delas deve responder a esse alvo. Essa decisão não é tomada apenas por regras fixas: uma rede neural, evoluída por um algoritmo genético próprio, aprende a refinar a escolha da torre e o instante do acionamento, sempre dentro dos limites que uma camada de regras explícitas (a heurística) já considera seguros. Esse aprendizado acontece em duas etapas bem separadas: antes da operação do sistema, em um ambiente de treinamento simulado, onde a rede evolui ao longo de muitas gerações de episódios de teste; e, depois de treinada, apenas na forma de inferência durante a execução real, sem qualquer ajuste adicional de seus parâmetros em tempo de operação. Antes de um genoma treinado ser promovido a essa condição de uso em produção, ele precisa alcançar um fitness médio comparável ao da heurística pura no mesmo regime de avaliação – na prática, dentro de poucos pontos percentuais de diferença (o genoma atualmente embutido ficou a 4% da heurística pura, ver Seção 2.2), e não apenas evoluir sem regredir. O problema central que este trabalho busca resolver é, portanto, como automatizar essa cadeia – detectar, localizar e selecionar a torre mais adequada a cada alvo – de forma auditável e replicável em laboratório, sem depender de sensores ou atuadores de uso industrial.

2 Desenvolvimento

2.1 Revisão da literatura e fundamentação teórica

Sistemas de detecção e rastreamento de alvos costumam recorrer a sensores como radar. Como alternativa, sensores de uso convencional, sensores domésticos podem ser combinados de modo que as limitações de um sejam compensadas pelas qualidades do outro. Sensores ultrassônicos, como o HC-SR04, medem distância com boa confiabilidade em curto alcance, mas não distinguem o tipo do obstáculo; câmeras, por sua vez, fornecem informação semântica, porém dependem das condições de iluminação e exigem maior poder de processamento (MAJONI et al., 2025). A fusão de sensores (*sensor fusion*) combina as duas modalidades para aproveitar essa complementaridade.

Majoni et al. (2025) descrevem uma arquitetura em que um Arduino Mega trata o sensoriamento ultrassônico de baixa latência, enquanto um Raspberry Pi processa as imagens da

câmera com uma rede YOLOv4-tiny, com comunicação serial entre os dois. Essa divisão de papéis é análoga à do RadarTorres, em que o Arduino realiza a aquisição e a aplicação desktop concentra o processamento. A versão de YOLO empregada no RadarTorres, porém, é mais recente que a do trabalho citado: o sistema usa o **YOLO26n** (Ultralytics, nano), não o tiny-YOLOv3/v4 de gerações anteriores. A diferença central é arquitetural – o YOLO26 tem cabeça de predição *one-to-one* em vez de *one-to-many*, de modo que cada objeto já gera uma única predição, eliminando a etapa de *Non-Maximum Suppression* (NMS) que os modelos antigos exigiam para descartar caixas duplicadas. Isso remove um algoritmo $O(n^2)$ por quadro e um componente de código inteiro para manter, o que foi decisivo dado que o RadarTorres roda em CPU, sem GPU dedicada garantida.

A literatura também mostra que sensores ultrassônicos acessíveis podem complementar outras fontes de sensoriamento em sistemas de rastreamento, com a precisão melhorada por predição baseada em filtro de Kalman e fusão com a distância medida pelos ultrassônicos (BAEK et al., 2020). O mesmo trabalho observa que as leituras ultrassônicas se tornam ruidosas em distâncias maiores, o que motiva o uso de filtragem.

Definidos o alvo e sua posição, resta decidir qual recurso deve responder. Na literatura de defesa, esse problema é tratado como avaliação de ameaças e atribuição de armas (*Threat Evaluation and Weapon Assignment*, TEWA), no qual as informações dos sensores alimentam as etapas de detecção, rastreamento e identificação, seguidas da avaliação da ameaça e da atribuição dos recursos. As abordagens desse tipo de problema costumam ser classificadas em ameaça por ameaça, em que os recursos são atribuídos sequencialmente, e multiameaça, em que o conjunto de ameaças é considerado simultaneamente.

O RadarTorres adota uma abordagem simples, por proximidade, aplicada alvo a alvo, com propósito exclusivamente demonstrativo (laser de baixa potência, LED ou simulação, sem armamento real); a abordagem multiameaça fica como trabalho futuro.

O refinamento dessa escolha por proximidade – e o instante de disparo – é feito por uma pequena rede neural *feedforward* (sinal que só percorre a rede em um sentido, da entrada à saída, sem laços de realimentação) com 7 entradas, 1 camada oculta de 8 neurônios, 2 saídas e ativação tanh (função que comprime a soma ponderada de cada neurônio no intervalo $[-1, 1]$). Os pesos dessa rede não são ajustados por retropropagação (o método usual de treino por gradiente em redes neurais), mas evoluídos por um algoritmo genético próprio (seleção por torneio, *crossover* uniforme, mutação gaussiana, elitismo), auditável e sem dependência de biblioteca de aprendizado de máquina. Such et al. (2017) mostram que algoritmos genéticos são uma alternativa competitiva ao treino por gradiente em tarefas de aprendizado por reforço, em redes de porte bem maior do que a aqui empregada (até quatro milhões de parâmetros, contra os 82 parâmetros da rede do RadarTorres). Esse resultado sustenta a escolha do algoritmo genético como método de treinamento viável para o RadarTorres, mas, por tratar de uma escala de rede muito maior, não sustenta por si só a conclusão de que aumentar o número de camadas ou neurônios aqui seria desnecessário; essa decisão, em vez disso, é justificada diretamente pelo experimento de hiperparâmetros da Seção 2.3.6. Por ora, a estratégia adotada para ampliar o desempenho do controlador prioriza o ajuste dos hiperparâmetros da busca evolutiva (tamanho de população, paralelização da avaliação de fitness, enriquecimento do sinal de recompensa), sem descartar, a priori, uma rede maior caso evidências futuras mostrem que a capacidade atual seja o fator limitante.

A arquitetura híbrida do controlador – heurística que restringe o espaço de ações antes da rede decidir, e volta a intervir depois, vetando o disparo – tem paralelo direto com duas técnicas da literatura de aprendizado por reforço. A primeira é o mascaramento de ações inválidas (*action masking*): Huang e Ontañón (2020) mostram, para algoritmos de gradiente de política, que mascarar o espaço de ações antes da amostragem corresponde a uma atualização de gradiente válida e se torna mais importante quanto maior a fração de ações inválidas – o que justifica tecnicamente a etapa em que a camada heurística do RadarTorres restringe as torres candidatas (nunca ocupada, nunca fora de alcance) antes de a rede neural escolher entre elas. A segunda é o *shielding*: Alshiekh et al. (2017) propõem um componente reativo (*shield*) que intercepta a ação escolhida pela política aprendida e a substitui quando ela violaria uma propriedade de segurança expressa em lógica temporal – papel equivalente ao do veto heurístico final do RadarTorres sobre o disparo, que nunca é delegado à rede. A atribuição de recursos defensivos a ameaças, por sua vez, é tratada na literatura de TEWA tanto no caso ameaça por ameaça quanto no caso multiameaça (NAEEM et al., 2009b, 2009a); o RadarTorres adota deliberadamente a abordagem mais simples (proximidade, alvo a alvo), com a multiameaça permanecendo como trabalho futuro já apontado anteriormente.

O impacto do treinamento virtual sobre a execução física do controlador é atenuado por uma decisão de projeto central: o mesmo controlador híbrido – heurística e rede neural combinadas – que decide no episódio simulado é o que decide em produção, consultado pelo algoritmo responsável por selecionar a torre a partir das leituras do Arduino real – não há conversão para um formato intermediário (ONNX, TensorFlow Lite) nem reimplementação em outra linguagem ou plataforma, fontes comuns de divergência entre o comportamento treinado e o executado em sistemas de aprendizado por reforço aplicados a robótica. O genoma evoluído é um vetor plano de pesos e vieses, embutido diretamente na aplicação por uma etapa automática de build, que grava o melhor genoma treinado no executável, de onde é carregado por um componente dedicado em tempo de execução; o que muda do ambiente de treino para a operação real é apenas a origem dos dados de entrada (leitura simulada ou leitura do Arduino), nunca a função de decisão em si. Os parâmetros que definem o episódio simulado – alcance de engajamento e certeza mínima para autorizar disparo – também deixaram de ser constantes fixas no código e passaram a ser lidos das mesmas configurações de alcance de mira e de limiar de certeza usadas pelo sistema real, de modo que recalibrar o alcance da câmera ou o limiar de certeza no painel de Configurações recalibra também o mundo de treino, reduzindo a divergência entre o que foi aprendido e o que será executado. Por fim, a rede neural nunca decide isoladamente: ela apenas pondera a escolha dentro do que a camada heurística determinística já considera válida (nunca uma torre ocupada, nunca fora de alcance), de modo que a arquitetura foi concebida para limitar o pior caso de uma generalização imperfeita do treino virtual para a física – no limite, o controlador regrediria ao comportamento puramente heurístico, nunca a uma decisão arbitrária; trata-se, porém, de uma propriedade de projeto ainda não submetida a validação em hardware físico real.

2.2 Procedimentos metodológicos

O percurso metodológico deste trabalho não seguiu o fluxo tradicional de especificação prévia seguida de implementação: o protótipo RadarTorres já existia, em desenvolvi-

mento incremental, antes do levantamento formal de requisitos. Optou-se, portanto, por catalogar os requisitos funcionais e não funcionais diretamente a partir do que já estava implementado no código-fonte da aplicação desktop e da documentação técnica já produzida ao longo do projeto, em vez de defini-los a priori em um documento de especificação anterior à codificação. O treinamento do controlador por neuroevolução, descrito na Seção 2.1, segue essa mesma lógica incremental: a arquitetura híbrida heurística+rede neural foi definida e ajustada em paralelo ao restante do sistema, não a partir de um plano de treinamento fechado desde o início.

Esse levantamento resultou em 25 requisitos funcionais (RF01–RF25) e 20 requisitos não funcionais (RNF01–RNF20), classificados segundo dois critérios objetivos: a **prioridade**, atribuída conforme a criticidade de cada requisito para a operação central do sistema (comunicação serial, rastreamento, seleção de torre, modos de operação e segurança recebem prioridade Alta; suporte administrativo e ferramental, Média; conveniências de interface, Baixa); e o **status**, que reflete o estado real de implementação constatado no código no momento do levantamento – Implementado, Parcial ou Planejado –, e não a intenção original de projeto. Cada requisito foi, além disso, rastreado até a classe ou método do código-fonte que o realiza, registro mantido em uma matriz de rastreabilidade dedicada, de modo que qualquer afirmação sobre o grau de implementação do protótipo possa ser verificada diretamente no código. Divergências encontradas entre o comportamento observado no código e a nomenclatura ou fluxo originalmente esperado também foram registradas explicitamente, em vez de omitidas ou corrigidas silenciosamente na documentação.

A parte física do RadarTorres (torres, motores, sensores montados em campo) ainda não foi implementada nem validada. Toda a validação funcional relatada neste trabalho foi realizada por simulação de software: a lógica de rastreamento de alvos e de seleção automática de torre foi exercitada de ponta a ponta sem depender de uma porta serial conectada a hardware real, o que impõe um limite claro ao escopo dos resultados relatados – eles atestam a corretude da lógica de decisão implementada em software, mas não substituem uma validação de campo com sensores e torres físicas, deixada como trabalho futuro.

A mesma ressalva se aplica, com reforço adicional, ao controlador treinado por neuroevolução: o mundo simulado em que os genomas são avaliados é uma aproximação idealizada do ambiente físico – os alvos nascem na borda de um círculo e avançam de forma quase radial, sem o ruído típico de leituras ultrassônicas reais; a certeza de identificação do alvo pela câmera é modelada como uma função sintética que cresce com o tempo de observação, e a probabilidade de errar o disparo é uma função assumida, crescente com a velocidade do alvo – nenhuma delas calibrada com dados de campo, já que o hardware ainda não foi montado. Essa distância entre simulação e realidade física não é apenas teórica: durante este trabalho, identificou-se e corrigiu-se uma falha em que a certeza de identificação do alvo era sorteada uma única vez, no nascimento, e nunca era atualizada – cerca de 70% dos alvos simulados nasciam permanentemente abaixo do limiar mínimo de disparo e jamais podiam ser abatidos, independentemente da qualidade do genoma, mascarando qualquer evolução real do controlador sob a aparência de estagnação. O episódio é citado aqui como evidência concreta de que uma premissa equivocada na simulação pode invalidar silenciosamente o processo de treinamento sem que isso seja percebido pelas métricas agregadas, reforçando a necessidade de validação cruzada

entre o comportamento do genoma treinado e o comportamento observado em campo, com hardware físico, antes de se confiar no controlador em um cenário de operação real – validação que permanece como trabalho futuro, nos mesmos termos já apontados para o restante do protótipo.

A Tabela 1 quantifica esse achado, a partir do histórico de treinamento persistido localmente (`historico_treinamento_geracoes` e `marcos_fitness_treinamento`, tabelas descritas na Seção 2.5).

Table 1: Fitness da população antes e depois da correção do bug de certeza estática (commit `a02d46a`), a partir do histórico de treinamento persistido localmente.

Métrica	**Antes (bug)**	**Depois (corrigido)**
Fitness médio da população	≈ 9,11 (min. 8,37, máx. 9,43; 51 blocos de 500 gerações)	134,17 (min. 121,31, máx. 150,60; amostra de 300 gerações)
Melhor fitness por bloco/geração	97,75 a 144,95 (média 119,62)	137,35 em média (122,81 a 155,47)
Taxa de cobertura dos alvos	não medida diretamente – ≈ 70% dos alvos permanentemente abaixo do limiar	100% (5/5 amostras pós-correção)
Genoma embutido no executável (atual)	–	geração 159, fitness 188,47

2.2.1 Ambiente computacional

Os experimentos controlados de hiperparâmetros relatados na Seção 2.3.6 e os tempos de treinamento ali citados (entre 14 e 75 segundos, para 300 gerações) foram executados em uma única máquina, cuja configuração de hardware e software é descrita na Tabela 2. O registro explícito desse ambiente é condição necessária para que os tempos medidos sejam interpretáveis: o laço de avaliação do algoritmo genético roda inteiramente sobre a CPU, sem qualquer aceleração por GPU – a placa de vídeo integrada da máquina de testes não possui suporte a CUDA nem é utilizada pelo `AlgoritmoGenetico` ou pelo `AmbienteTreinamento`, de modo que o tempo de treinamento depende do desempenho do processador e não da placa gráfica. Reportar hardware, sistema operacional e *runtime* é, portanto, pré-condição para que os tempos absolutos citados na Seção 2.3.6 (o experimento original, de seis entradas) sejam reproduzíveis ou comparáveis em outra máquina: um processador mais rápido, mais núcleos ou memória disponível alteram os segundos medidos sem alterar a lógica avaliada, e a ausência de uso de GPU explica por que o tempo escala com o tamanho da população (mais indivíduos avaliados por geração) e quase não escala com o tamanho da rede neural (Seção 2.3.6, parágrafo de custo computacional) – o gargalo é o número de episódios simulados em CPU, não a propagação da rede, pequena demais para pesar no tempo total.

Essa descrição de ambiente cobre especificamente os onze experimentos de hiperparâmetros da Seção 2.3.6, executados nesta máquina e nesta configuração de *software* no mesmo dia.

Table 2: Ambiente computacional dos experimentos de hiperparâmetros (Seção 2.3.6).

Componente	**Especificação**
Processador	Intel® Core™ i5-4200U @ 1,60 GHz, 2 núcleos físicos / 4 threads, clock máximo de 2,30 GHz
Memória RAM	16 GB
Placa de vídeo	Intel HD Graphics Family (integrada), 1024 MB de memória compartilhada, sem GPU dedicada e sem aceleração CUDA
Sistema operacional	Microsoft Windows 11 Home, versão 10.0.22631 (*build* 22631), 64 bits
Runtime	.NET SDK 9.0.320, *runtime* Microsoft.WindowsDesktop.App 9.0.x, *target framework* `net9.0-windows`
Principais dependências	Microsoft.Data.Sqlite 9.0.0, Microsoft.Extensions.* 9.0.0, Microsoft.ML.OnnxRuntime 1.20.1, AForge.Video / AForge.Video.DirectShow 2.2.5, System.Drawing.Common 9.0.0, PDFsharp 6.2.4, System.IO.Ports 9.0.0

Ela **não** se estende ao treinamento mais longo que produziu o genoma de produção embutido no executável (`Resources/ControladorTreinado/genoma-controlador.json`): o histórico de treinamento persistido localmente (Tabela 1 e Seção 2.5) registra gerações, fitness e horários, mas não qual máquina executou cada sessão de treinamento, de modo que não é possível afirmar que o genoma de produção foi gerado neste mesmo ambiente de hardware. Essa lacuna de rastreamento é registrada aqui como limitação explícita e como trabalho futuro: o `AmbienteTreinamento` poderia passar a persistir, junto do histórico de gerações, uma identificação da máquina de execução (processador, núcleos e presença ou não de GPU), tornando toda sessão de treinamento – e não apenas os experimentos controlados desta seção – igualmente rastreável e reproduzível.

Como verificação adicional de portabilidade, o executável foi também inicializado em uma segunda máquina, de configuração distinta da descrita na Tabela 2: processador Intel® Core™ i5-8400 @ 2,80 GHz (6 núcleos físicos / 6 threads), 7,83 GB de memória RAM, placa de vídeo integrada Intel UHD Graphics 630 (sem GPU dedicada e sem aceleração CUDA), Microsoft Windows 11 Pro (versão 10.0.26300, 64 bits) e *runtime* .NET SDK 9.0.316 com Microsoft.WindowsDesktop.App 9.0.18. O teste consistiu na inicialização do aplicativo nesse ambiente, sem execução de rotinas adicionais: a janela principal abriu normalmente, sem erros. Esse resultado confirma apenas que a aplicação inicializa corretamente em outra configuração de hardware e versão do sistema operacional – não substitui, e não deve ser confundido com, os experimentos controlados de hiperparâmetros da Seção 2.3.6, que permanecem associados exclusivamente à máquina da Tabela 2.

2.3 Projeto e implementação

2.3.1 Módulo de detecção de alvos

O módulo de detecção utiliza dois sensores ultrassônicos HC-SR04 instalados de costas um para o outro no topo de um mastro, na torre central. O mastro é movido por um motor de passo 28BYJ-48 que oscila 180°, de modo que, em conjunto, os dois sensores cobrem uma varredura de 360°. Cada sensor emite pulsos de 40 kHz e calcula a distância pelo atraso do eco, segundo a relação $d = \frac{v \cdot \Delta t}{2}$, em que v é a velocidade do som no ar (cerca de 343 m/s à temperatura ambiente). A cada leitura, o Arduino envia à aplicação desktop, pela porta serial, o ângulo e a distância em protocolo de texto simples, que a aplicação converte em posição cartesiana e exibe no radar circular de quatro quadrantes.

O protocolo é textual em ASCII, transportado pela porta serial USB a 9600 bauds por padrão, com cada mensagem terminada por `\n` e composta por um tipo em maiúsculas seguido de pares chave-valor separados por ponto e vírgula. Uma leitura de alvo chega na forma `TARGET;ID=1;ANGLE=45;DIST=2.80`, em que `ANGLE` é dado em graus no intervalo $[0, 360)$ e `DIST` em metros. O ângulo segue a convenção de bússola – 0° apontando para o Norte, sentido horário –, escolhida por ser a leitura mais intuitiva para quem opera um radar, e a conversão para coordenadas cartesianas de mundo, feita por uma rotina dedicada de conversão de coordenadas polares para cartesianas, é dada por $x = d \cdot \sin\theta$ e $y = d \cdot \cos\theta$, com θ em radianos. A partir do par (x, y), o quadrante do alvo é determinado pelos sinais das duas coordenadas (por um algoritmo auxiliar de identificação de quadrante) e a distância até cada torre candidata pela distância euclidiana clássica (calculada por uma rotina dedicada de cálculo de distância). Leituras com campos ausentes, não numéricos, com ângulo fora do intervalo válido ou com distância negativa são convertidas em mensagens de erro registradas no console de eventos, nunca em exceções – falhas de sensor ou de cabo produzem degradação graciosa, e não interrupção da aplicação.

2.3.2 Módulo de visão

O módulo de visão utiliza quatro câmeras de 720p conectadas por USB ao computador, associadas aos quadrantes do radar, e processa os quadros de vídeo (*frames*) para classificar o alvo por sua cor, formato e tamanho. Cada quadro capturado é entregue já decodificado à detecção, que executa localmente um modelo YOLO26, na variante nano, em formato ONNX, para propor as regiões candidatas da imagem. Cada candidata é então comparada ao perfil do alvo previamente cadastrado – definido por cor, formato e tamanho, e opcionalmente por fotos de referência recortadas no objeto de interesse. Havendo foto de referência, a comparação combina um histograma de matiz e saturação com uma grade espacial de cor média, usados como assinatura do objeto; na ausência de foto, o critério recai sobre a fração de pixels com a cor cadastrada e sobre a classificação de formato. Quando o detector não propõe nenhuma caixa com a cor do perfil, as regiões candidatas passam a ser propostas por segmentação de cor, como último recurso na ordenação de candidatos.

O envio das informações resultantes ao Arduino, para guiar o apontamento da torre (movimentos de pan e tilt), foi projetado mas ainda não está implementado: o protocolo serial atual não possui comando de apontamento, e a fusão entre a posição estimada pela

visão e a distância medida pelos sensores ultrassônicos permanece como etapa planejada. Como o hardware ainda não foi montado, a validação funcional descrita neste trabalho foi feita por simulação, e nenhum resultado de campo é relatado.

2.3.3 Fluxo operacional

Em operação, o sistema segue estas etapas: (1) o Arduino varre o ambiente com os dois sensores ultrassônicos e envia ângulo e distância à aplicação; (2) a aplicação converte cada leitura em posição cartesiana, determina o quadrante, exibe o alvo no radar e registra a detecção; (3) em paralelo, as câmeras fornecem quadros de vídeo, e o módulo de visão classifica o alvo por cor, formato e tamanho; (4) a aplicação seleciona, entre as torres armadas disponíveis, a mais adequada para o alvo, priorizando as do mesmo quadrante e, entre elas, a de menor distância euclidiana (por proximidade, alvo a alvo); (5) o comando é enviado ao Arduino, que aciona a torre selecionada; (6) o acionamento demonstrativo (laser de baixa potência ou LED) só ocorre respeitando a distância mínima de segurança e as demais regras de segurança configuráveis do sistema; (7) toda ação executada é registrada em auditoria.

A seleção de torre e o acionamento dependem do modo de operação escolhido pelo operador, em um modelo de três níveis: no primeiro, os alvos são apenas detectados e exibidos; no segundo, acrescenta-se a seleção automática da torre, sem acionamento; no terceiro, acrescenta-se o acionamento demonstrativo automático. Quando a regra de segurança reprova um acionamento, ele é bloqueado e o motivo é registrado no console de eventos.

2.3.4 Controlador neuroevolutivo: heurística e rede neural combinadas

O passo (4) do fluxo operacional – escolher, entre as torres armadas disponíveis, a mais adequada para cada alvo, e decidir o instante do acionamento – é resolvido no RadarTorres pela combinação de dois mecanismos de natureza distinta: uma **heurística** determinística, escrita como regras explícitas de engenharia, e uma **rede neural** *feedforward*, cujo comportamento é aprendido por um algoritmo evolutivo próprio, sem biblioteca de aprendizado de máquina. Ambos coexistem no mesmo controlador híbrido, e é essa combinação – não cada mecanismo isoladamente – que determina tanto o comportamento em produção quanto o que é efetivamente treinado.

A heurística é um conjunto de regras fixas, escritas e revisáveis por engenharia, cujo resultado para uma mesma entrada é sempre o mesmo. No RadarTorres ela resolve, em cascata, três camadas de prioridade na seleção da torre: primeiro considera apenas as torres livres do próprio quadrante do alvo; na ausência de candidata, estende-se às torres de quadrante adjacente cujo quadrante de origem não tenha outro alvo concorrente; por fim, na ausência de qualquer uma das duas, recorre a qualquer torre livre restante, de modo que nenhum alvo fique sem cobertura. Cada camada carrega um fator numérico (1,0; 0,5; ou 0,0) que soma-se diretamente à pontuação da candidata. A heurística também impõe quatro regras de segurança e refinamento, cada uma ativável individualmente no painel de Configurações: anti-oscilação, que evita trocar de torre por uma diferença de pontuação dentro de uma margem de empate; priorização por urgência, que permite a um alvo muito próximo da base "tomar" a torre de um alvo mais distante; balanceamento

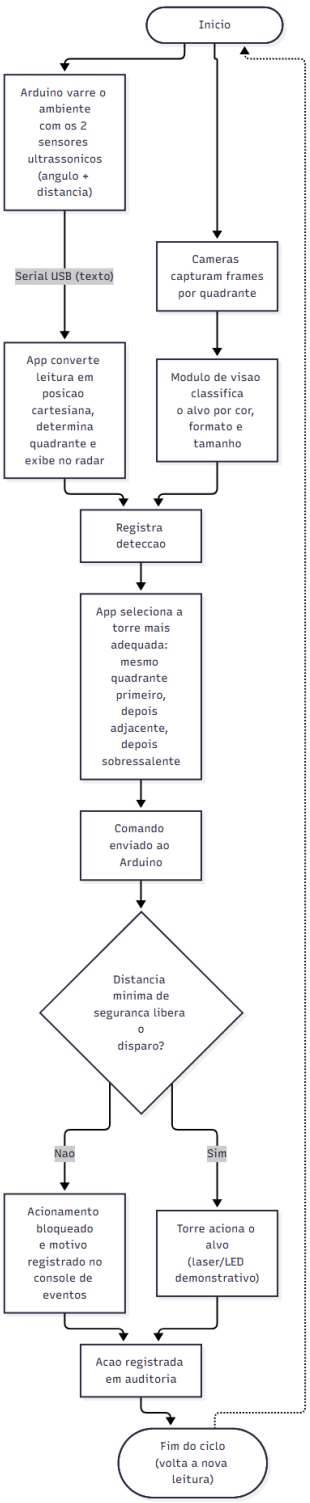


Figure 1: Fluxograma do fluxo operacional do RadarTorres, da leitura dos sensores ao registro em auditoria (dos autores).

de carga, que prefere, entre torres empatadas, a usada com menor frequência na sessão; e predição de trajetória, que projeta a posição futura do alvo para calcular o ângulo de mira com antecedência de poucos décimos de segundo. Por fim, a heurística impõe um limite absoluto ao disparo, nunca delegado à rede: um limiar mínimo de certeza de identificação do alvo (`LimiarConfiancaDisparoPercentual`), abaixo do qual o disparo é recusado independentemente de qualquer outra avaliação.

A rede neural, por sua vez, não decide essas regras de segurança nem a cascata de camadas – ela decide apenas a escolha fina *dentro* do espaço já validado pela heurística. É uma rede *feedforward* (sinal que só percorre a rede em um sentido, da entrada à saída, sem laços de realimentação) totalmente conectada, implementada do zero (sem biblioteca de aprendizado de máquina). Ajustar caso a biblioteca de neuroevolução funcione (biblioteca em teste: [NOME
A rede tem arquitetura de 7 entradas, uma camada oculta de 8 neurônios e 2 saídas, com ativação tanh (função que comprime a soma ponderada de cada neurônio no intervalo $[-1, 1]$) em todas as camadas. As sete entradas – todas normalizadas no intervalo $[0, 1]$ ou equivalente – são a distância normalizada ao alvo, dois indicadores binários de quadrante (mesmo quadrante e quadrante adjacente), o ângulo de mira normalizado, a velocidade normalizada do alvo, o tempo normalizado em que o alvo está sendo detectado sem ter sido abatido (satura em 1,0 a partir de 3 s de observação contínua – ver Seção 2.3.6) e o próprio fator da camada heurística vigente, usado como sinal de contexto. As duas saídas são um ajuste de pontuação, somado à pontuação heurística da candidata, e um valor de confiança de disparo, cujo sinal positivo autoriza o acionamento. Os pesos e vieses da rede – 82 parâmetros, no total, para essa arquitetura – não são ajustados por retropropagação, mas evoluídos por um algoritmo genético: cada genoma é um vetor plano desses 82 números, a população evolui por seleção por torneio, *crossover* uniforme, mutação gaussiana (perturbação de cada peso por um valor aleatório de distribuição normal, preferida a outras formas de mutação por manter a maioria das alterações pequenas e permitir ajuste fino sem descartar o que o genoma já aprendeu – detalhamento comparativo com outros tipos de mutação em `ApoioDeApresentacao.md`) e elitismo, e o genoma final é embutido na aplicação como um vetor de pesos fixo, sem qualquer etapa de treinamento em tempo de execução.

Impacto de acrescentar uma entrada. A sétima entrada (tempo detectado sem abate) não fazia parte do projeto original – a rede nasceu com seis entradas e cinco indicadores de contexto, e essa sétima foi incluída posteriormente para dar à rede um sinal que faltava: por quanto tempo um alvo específico já vem sendo observado sem ter sido abatido, de modo que a evolução pudesse aprender a priorizar alvos "esquecidos" em vez de tratar todos como igualmente urgentes. Mudar a contagem de entradas de uma rede já evoluída, porém, não é uma alteração isolada, e vale registrar os quatro impactos concretos que ela trouxe, cada um observado diretamente neste trabalho e não apenas previsto em teoria.

O primeiro impacto é no tamanho da rede: de 74 para 82 parâmetros (aumento de **8 pesos**, um por neurônio da camada oculta, ligando a nova entrada a cada um deles – a camada oculta continuou com 8 neurônios e as duas saídas não mudaram). É um acréscimo de 10,8% no genoma, mas, como a Seção 2.3.6 confirma experimentalmente, o tempo de treinamento continua dominado pela quantidade de episódios simulados por geração (função do tamanho da *população*), não pelo tamanho da rede avaliada em cada episódio

– oito pesos adicionais são imperceptíveis nesse custo, e nenhuma das onze execuções da grade mostrou um tempo de treinamento mensuravelmente maior apenas por ter mais ocultos do que antes.

O segundo impacto, esse sim severo, é a **incompatibilidade retroativa total**: qualquer genoma evoluído sob a arquitetura antiga (seis entradas, 74 pesos) deixou de ser utilizável, porque `RedeNeural.Ativar` exige que o vetor de entradas tenha exatamente o tamanho declarado pela arquitetura – não há conversão possível entre um genoma de 74 e um de 82 números, nem forma de "completar" o genoma antigo com os oito pesos que faltam sem reavaliar do zero o que a evolução faria com eles. Isso invalidou, de uma só vez, o controlador de produção já embutido no executável (`Resources/ControladorTreinado/genoma-controlador` treinado sob a arquitetura antiga) e todas as 11 execuções da Seção 2.3.6 documentadas anteriormente – ambos precisaram ser retreinados do zero sob a nova arquitetura (e, no mesmo retreinamento, sob o ambiente também endurecido, ver próximo parágrafo) para voltarem a valer. Esse acoplamento rígido entre arquitetura e genoma é uma propriedade conhecida de redes neurais simples sem camadas de embedding ou *padding*: cada entrada nova é, na prática, uma mudança de *breaking change* no formato do genoma, nunca uma extensão compatível com versões anteriores.

O terceiro impacto é operacional, não só experimental: como o próprio executável em produção (`TowerSelectionService`) chama a mesma função estática `ControladorHibrido.ConstruirE` usada pelo treinamento – por design, para garantir que a rede treinada receba em campo exatamente a mesma entrada que recebeu durante a evolução –, acrescentar uma entrada obriga a atualizar os DOIS lados ao mesmo tempo, simulação e sistema real, e a localizar, no sistema real, um valor de verdade que corresponda ao conceito simulado. Neste caso a atualização foi simples porque a grandeza já existia no sistema real: `Target.Timestamp` já registrava o instante da primeira detecção de cada alvo (usado para outros fins), bastando calcular `(DateTime.UtcNow - Target.Timestamp)` no ponto de decisão. Esse não é o caso geral – uma entrada hipotética sem contrapartida instrumentada no hardware real teria exigido adicionar sensoriamento ou lógica nova ao sistema físico antes de a rede treinada em simulação poder ser usada de verdade, e não apenas ajustar uma fórmula de normalização.

O quarto impacto foi preventivo: para que a incompatibilidade do segundo impacto não se manifestasse como uma exceção não tratada na primeira seleção de torre real (o que teria travado o aplicativo em produção com o genoma antigo ainda embutido, entre o momento em que o código mudou e o momento em que um novo genoma foi treinado e reembutido), foi adicionada uma verificação explícita de compatibilidade em `ControladorTreinadoProvider`: antes de instanciar a rede a partir do genoma embutido, o número de entradas declarado no arquivo é comparado ao `ControladorHibrido.QuantidadeEntradas` atual, e qualquer divergência faz o sistema degradar, deliberadamente, para a heurística pura – o mesmo comportamento seguro de "nunca travar, na pior hipótese operar só com a heurística" já descrito para as demais camadas de segurança do controlador híbrido. Esse guarda não existia antes da mudança de arquitetura; passou a existir *por causa* dela, e é hoje o que torna seguro acrescentar uma oitava, nona ou décima entrada no futuro sem risco de travar o aplicativo de alguém que ainda não retreinou.

Uma ressalva metodológica fecha esta discussão: a entrada nova e o endurecimento do ambiente de treino (ver próximo parágrafo) foram introduzidos na mesma leva de mudanças e avaliados juntos nas onze execuções da Seção 2.3.6, de modo que este trabalho

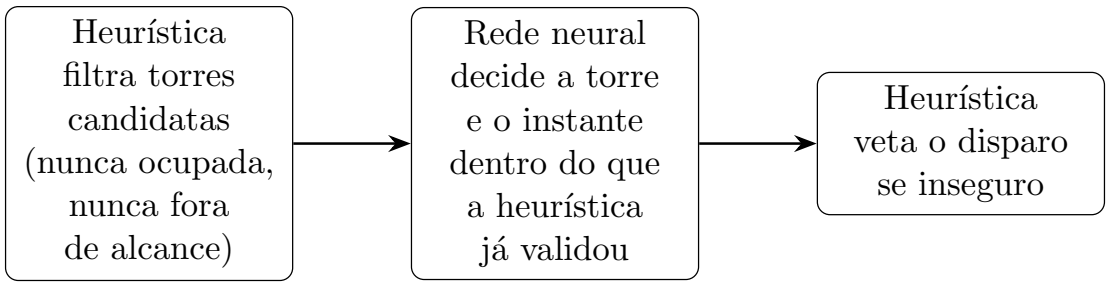
não isola o efeito de ter mais uma entrada do efeito de o ambiente ter ficado mais difícil – os dados aqui não permitem afirmar que a sétima entrada, por si só, melhorou (ou piorou) a qualidade do controlador. Uma comparação controlada, com a sétima entrada mas no ambiente ANTIGO (sem invasão de torre nem teto por quadrante), ou vice-versa, é o experimento de ablação natural para isolar essa pergunta, e fica registrada aqui como trabalho futuro específico, distinto da ablação heurística-*vs*-rede já conduzida (Seção 2.3.6) e distinto também da repetição com múltiplas sementes já apontada como limitação geral.

A combinação entre os dois mecanismos é, portanto, sequencial e hierárquica, nunca uma mistura indiferenciada: a heurística primeiro filtra quais torres são elegíveis e calcula as *features* de entrada (quadrante, ângulo, fator de camada); a rede neural, em seguida, pondera essa pontuação dentro do subconjunto já validado, decidindo qual candidata específica vence o empate fino que a heurística, por si só, não resolveria com a mesma sensibilidade; e, por fim, a heurística volta a atuar como filtro de segurança sobre a decisão de disparo, vetando qualquer autorização da rede que viole a certeza mínima de identificação do alvo. Essa hierarquia tem uma consequência deliberada para o pior caso: uma generalização imperfeita da rede nunca produz uma decisão arbitrária, porque toda saída da rede continua restrita ao espaço de candidatas e ao conjunto de disparos já pré-aprovados pela heurística – no limite, o sistema regride ao comportamento puramente heurístico, nunca a um erro fora de controle.

Esse acoplamento se reflete diretamente no treinamento do controlador. O algoritmo genético não avalia a rede neural isolada: avalia o comportamento do `ControladorHibrido` completo – heurística e rede juntas – em episódios simulados (45 s por padrão; o genoma de produção atual foi retreinado sem limite de duração, ver Seção 2.3.6), com as torres reais do projeto, alvos nascendo na borda de um mundo circular e avançando em direção à base. O fitness de cada genoma soma recompensas por abate (maiores quanto mais longe da base o alvo é interceptado) e penalidades por alvo que escapa ou cruza um ponto de perda sem decisão, de modo que o genoma é pressionado a aprender a operar bem *dentro* das restrições que a heurística já impõe, e não a contorná-las – não há genoma "vencedor" que dispare abaixo da certeza mínima, porque essa decisão nunca chega a ser avaliada como tal pelo ambiente de treino.

O ambiente de treino foi posteriormente endurecido em quatro pontos, para aproximar melhor a simulação de um cenário de saturação real. Primeiro, o teto de alvos simultâneos deixou de ser global e passou a valer por quadrante (`MaxAlvosPorQuadrante`): cada um dos quatro quadrantes pode estar no próprio limite ao mesmo tempo, em vez de um único teto compartilhado por todo o mundo. Segundo, o intervalo entre nascimentos de alvo encolhe ao longo do episódio – uma escalada de dificuldade que faz os alvos chegarem cada vez mais rápido conforme o tempo passa, em vez de a uma taxa constante do início ao fim. Terceiro, um alvo ainda ativo que chega perto demais de QUALQUER torre (não apenas da base) agora rompe a defesa e encerra o episódio imediatamente, descartando toda pontuação futura possível – distinto da fuga pela base, que só penaliza e deixa o episódio continuar. Quarto, ao fim de cada episódio o fitness é também penalizado pelo desvio padrão do tempo em que cada torre ficou comprometida com algum alvo: sobrecarregar sempre a mesma torre enquanto outra permanece ociosa agora custa pontos, mesmo que o total de abates tenha sido bom – uma medida explícita de gestão de recursos, e não só de taxa de abate. Essa mesma dependência, no entanto, também é o ponto mais sensível do processo: como o fitness

depende inteiramente do comportamento simulado, qualquer premissa equivocada no ambiente de treino se propaga, silenciosamente, para o genoma evoluído. Foi exatamente esse o caso identificado e corrigido neste trabalho – a certeza de identificação do alvo era sorteada uma única vez, no nascimento, e nunca era atualizada, de modo que cerca de 70% dos alvos simulados nasciam permanentemente abaixo do limiar mínimo de disparo e jamais podiam ser abatidos, independentemente da qualidade do genoma, o que mascarava qualquer evolução real sob a aparência de estagnação. A correção – fazer a certeza crescer com o tempo de observação do alvo, como ocorreria com uma câmera real acumulando quadros – e o alinhamento posterior dos parâmetros de alcance de mira e de limiar de certeza usados no treino com os mesmos valores configuráveis em produção (`ConfiguracoesAplicativo.AlcanceMiraMetros`, `LimiarConfiancaDisparoPercentual`) ilustram, em conjunto, por que a fidelidade do ambiente simulado ao comportamento do sistema real é, para um controlador neuroevolutivo, tão determinante para o resultado do treinamento quanto a própria arquitetura da rede ou os hiperparâmetros da busca genética.



`ControladorHibrido` – a mesma classe decide no treino simulado e na operação real

Figure 2: Fluxo de decisão do controlador híbrido: a heurística determinística filtra e, ao final, veta; a rede neural decide apenas dentro do espaço já validado (dos autores).

2.3.5 Regras heurísticas que regem o sistema

A Seção anterior descreveu a heurística em termos do seu papel (filtra, depois veta) e da cascata de três camadas (próprio quadrante, adjacente, sobressalente). Esta seção cataloga, em detalhe e numeradas, TODAS as regras heurísticas implementadas em código que efetivamente decidem qual torre persegue cada alvo e se um disparo é ou não autorizado – as mesmas regras que operam tanto na simulação de treino (`ControladorHibrido`/`AmbienteTreinament` quanto no sistema real (`TowerSelectionService`/`FireControlService`), já que a primeira é escrita para espelhar fielmente a segunda (Seção 2.3.4).

Cascata de elegibilidade (três camadas). Antes de qualquer pontuação, as torres livres são agrupadas em camadas, e só a primeira camada não-vazia participa da disputa:

1. **Torre do próprio quadrante** (`fatorCamada = 1,0`): se alguma torre cujo quadrante de cobertura coincide com o do alvo está livre, só elas concorrem.
2. **Torre de quadrante adjacente** (`fatorCamada = 0,5`): só entra em jogo se a camada 1 estiver vazia – e, dentro dela, só concorrem torres cujo PRÓPRIO quadrante

não tenha neste momento outro alvo ativo concorrente (não ajuda quem precisa de si mesma em casa).

3. **Torre sobressalente/qualquer livre** (`fatorCamada = 0,0`): cobertura de último recurso, só quando as camadas 1 e 2 estão ambas vazias – qualquer torre livre do sistema, mesmo diagonalmente oposta ao alvo, é melhor do que deixá-lo sem nenhuma torre perseguindo.

Se nenhuma torre está livre em NENHUMA camada ("sobrecarregada"), o alvo fica sem torre selecionada até alguma liberar – nunca há fila de espera ou reserva antecipada. Dentro da camada escolhida, a pontuação de cada candidata é a combinação heurística+rede descrita na seção anterior (`ControladorHibrido.Pontuar`), ou simplesmente a menor distância se nenhum controlador treinado estiver embutido (heurística pura de fábrica).

Cinco regras de refinamento (`TowerSelectionService`), as quatro primeiras desligáveis individualmente na tela de Configurações, a quinta sempre ativa. Aplicadas depois da cascata de camadas, nesta ordem lógica:

1. **Anti-oscilação.** Se a torre JÁ comprometida com o alvo continua sendo uma candidata válida e sua pontuação está dentro de uma margem de empate da vencedora (0,3 m sem rede, ou 0,1 ponto com rede embutida), a torre atual é mantida – evita trocar de torre por uma diferença de pontuação irrelevante, causada por ruído de leitura, quando a torre atual ainda serve bem.

2. **Priorização por urgência.** Um alvo é considerado "urgente" quando sua distância cai abaixo de 1,5× a distância mínima de segurança configurada. Um alvo urgente pode TOMAR a torre de outro alvo que esteja claramente mais longe (distância do outro maior que a do urgente mais 1 m E fora da própria zona de urgência) – o alvo que perdeu a torre não fica descoberto para sempre, só procura outra candidata na próxima leitura.

3. **Balanceamento de carga.** Entre as candidatas praticamente empatadas com a vencedora (mesma margem da regra 1), prefere a torre usada com MENOS frequência nesta sessão – contador reiniciado a cada execução do aplicativo, de propósito: é um critério de desempate do momento, não uma métrica histórica persistida.

4. **Predição de trajetória.** O ângulo de mira aponta para onde o alvo estará 0,3 s no futuro, projetado a partir da velocidade estimada (zero no primeiro instante de detecção, sem histórico ainda) – não para a posição atual. Afeta só a mira exibida/calculada, nunca a escolha de QUAL torre.

5. **Ajuda de torre adjacente contra alvo rápido (sempre ativa).** Alvos com velocidade igual ou acima do limiar configurado de "alvo rápido" recebem, além da torre principal, QUALQUER torre de quadrante adjacente a até 2 m de distância do alvo – mesmo fora do alcance normal de câmera – como reforço de disparo. Um segundo canhão reduz a chance de o alvo escapar de um tiro único, já que a probabilidade de erro cresce com a velocidade (`Helpers.CalculadoraErroDisparo`). É a única das cinco regras sem interruptor na tela de Configurações; só o limiar de velocidade que a aciona é ajustável.

Uma regra adicional, implícita e sempre ativa, nunca listada entre as cinco acima porque não é uma preferência e sim uma restrição física: nenhuma torre é candidata, em nenhuma camada, se outra torre qualquer estiver posicionada geometricamente entre ela e o alvo dentro de um corredor estreito de 0,3 m – ela bloquearia o próprio tiro.

Vetos finais, sempre ativos, no momento da autorização de disparo (`FireControlService.Autl` Mesmo com uma torre já selecionada, nenhum comando de acionamento sai do sistema sem passar por TODOS estes critérios, verificados nesta ordem e reavaliados a cada tentativa (uma torre selecionada agora pode ser vetada mais tarde, se a condição mudar entre a seleção e o disparo):

1. O alvo precisa continuar ativo (não pode ter saído de cena entre a seleção e a tentativa de disparo).
2. O sistema precisa estar no modo operacional `Disparar` – qualquer outro modo bloqueia incondicionalmente.
3. A certeza de identificação do alvo (percentual de confiança acumulado pela câmera) precisa ser maior ou igual ao limiar mínimo configurado (`LimiarConfiancaDisparoPercentual`) – nunca abaixo do piso absoluto do sistema.
4. A torre específica que está tentando disparar precisa constar na lista de torres efetivamente engajando aquele alvo (`Target.EngagingTowers`) – a mesma lista calculada pela regra 5; uma torre pode ter sido selecionada como "principal" e ainda assim não ter linha de tiro livre neste instante.

Vencidos os quatro critérios, o disparo em si – acertar ou errar o alvo – continua sendo probabilístico e cresce com a velocidade do alvo (mesma fórmula usada para calibrar o ambiente de treino, `CalculadoraErroDisparo`), nunca determinístico só por ter sido autorizado.

Uma regra adicional do controlador híbrido vale apenas durante a simulação de treino, nunca em produção, porque em produção o INSTANTE do disparo é sempre decidido manualmente por um operador, e não pelo controlador: o mesmo limiar mínimo de certeza da regra 3 acima, aplicado contra a certeza simulada do alvo em vez da real. Existe só para a avaliação de fitness do algoritmo genético corresponder ao que o sistema real, de fato, permite – não é uma regra "a mais" que o sistema real desconheça.

2.3.6 Análise experimental de hiperparâmetros

Fitness, termo-padrão da literatura de algoritmos genéticos (de onde o RadarTorres o herda), é a nota numérica que mede o quão bem um genoma específico (um vetor de pesos da rede) se comportou em um episódio simulado – soma de recompensas por abate e penalidades por fuga, invasão de torre e recusa de disparo (ver Seção 2.3.4). Um fitness que continua subindo entre gerações indica que a seleção, o *crossover* e a mutação estão de fato encontrando genomas melhores que os da geração anterior; um fitness estagnado indica que a busca chegou a um patamar (ótimo local ou teto do que a arquitetura/ambiente permite) e não está mais progredindo com as gerações restantes.

Para justificar tecnicamente a escolha dos hiperparâmetros do algoritmo genético com dados reais de treinamento, em vez de apenas por convenção, foram conduzidos experimentos controlados em que se varia *um* hiperparâmetro por vez, mantendo todos os demais fixos. Foram testados três tamanhos de população (30, 60 e 120 indivíduos) combinados com três arquiteturas de rede (4, 8 e 16 neurônios na camada oculta, mantendo fixas as 7 entradas e as 2 saídas descritas na Seção 2.3.4 – grade completa 3×3, nove execuções) e, isolando a taxa de mutação com população e arquitetura fixas na configuração de produção, duas execuções adicionais com 0,04 e 0,16 (metade e dobro do valor padrão de 0,08, já coberto pela grade), totalizando onze execuções independentes.

Cada configuração foi treinada isoladamente por exatamente 300 gerações, com a mesma função de fitness, os mesmos cenários táticos obrigatórios (`CatalogoCenarios.TodosComScript`) e o mesmo ambiente de avaliação do treinamento padrão – já no ambiente endurecido descrito ao final da Seção 2.3.4 (teto por quadrante, escalada de dificuldade, invasão de torre encerrando o episódio e penalidade de desequilíbrio entre torres). Todas as execuções usaram o mesmo `AmbienteTreinamento` e o mesmo `AlgoritmoGenetico` que treinam o controlador em produção, sem nenhum dado simulado ou estimado manualmente, rodadas em lote único e sequencial numa única máquina – não a da Tabela 2, e sim a segunda máquina descrita no parágrafo de portabilidade ao final da Seção 2.2.1 (processador Intel Core i5-8400 @ 2,80 GHz, 6 núcleos, Windows 11 Pro, .NET SDK 9.0.316) – para que os tempos cronometrados fossem comparáveis entre si. Por serem de uma máquina diferente, os tempos desta seção **não** devem ser comparados diretamente com os 14 a 75 segundos citados na Seção 2.2.1 para o experimento original (seis entradas): só as comparações *dentro* da Tabela 3, entre si, são válidas. O resultado de cada configuração – população, neurônios ocultos, taxa de mutação, fitness final, geração em que esse fitness foi alcançado pela primeira vez, curva de convergência nas gerações 25/50/100/200 e tempo de treinamento – está listado na Tabela 3 e exportado em `tools/RadarTorres.Experimentos/Resultados/experimentos_hiperparametros_7entradas.json`.

Table 3: As onze execuções da grade de hiperparâmetros, já com a arquitetura de 7 entradas e o ambiente endurecido. Fitness final e geração do pico idênticos (a menos de 0,02 ponto) em todas as linhas – ver discussão abaixo.

Pop.	Ocultos	Mutação	Fitness final	Pico (ger.)	Tempo (s)
30	4	0,08	68,33	222	3,4
30	8	0,08	68,34	222	3,3
30	16	0,08	68,34	222	3,6
60	4	0,08	68,34	222	6,3
60	8	0,08	68,34	222	8,2
60	16	0,08	68,34	222	7,9
120	4	0,08	68,34	222	11,3
120	8	0,08	68,34	222	13,4
120	16	0,08	68,34	222	20,5
60	8	0,04	68,34	222	6,9
60	8	0,16	68,34	222	6,3

Efeito sobre a qualidade final do controlador: o teto desapareceu por completo. A Tabela 3 substitui integralmente a Figura de fitness final da versão anterior deste experimento (seis entradas, ambiente sem invasão de torre nem penalidade de desequilíbrio): naquela versão, as nove combinações de população e arquitetura terminavam numa faixa estreita mas ainda perceptível, de 55,59 a 57,27 (variação de 3%). Com o ambiente endurecido e a arquitetura de sete entradas, essa faixa praticamente desaparece – as onze execuções terminam entre 68,33 e 68,34, uma variação de 0,01 ponto, e **todas atingem esse valor exatamente na mesma geração, 222**, independentemente de população (30 a 120), arquitetura (4 a 16 neurônios ocultos) ou taxa de mutação (0,04 a 0,16). Isso não é o mesmo resultado do experimento anterior levado ao extremo por acaso: é uma mudança estrutural na tarefa. Os cenários táticos obrigatórios (`CatalogoCenarios.TodosComScript`) são um conjunto pequeno e fixo de situações determinísticas; sob o ambiente endurecido, há um teto de pontuação alcançável nesse conjunto fixo de cenários que qualquer genoma minimamente competente atinge com folga antes mesmo de terminar as 300 gerações – a partir daí, não existe mais recompensa disponível para diferenciar um hiperparâmetro do outro, e a busca genética simplesmente para de ter o que otimizar. Em outras palavras, o endurecimento do ambiente tornou a bateria de cenários fixos *mais fácil de saturar*, não mais difícil de aprender – o oposto do que se esperaria de regras mais punitivas, e evidência de que o verdadeiro desafio do ambiente atual está no treinamento com cenários aleatórios (Seção 2.5), não nesta bateria fixa historicamente usada para comparar hiperparâmetros.

Efeito sobre a velocidade de evolução: ainda existe, mas só antes do teto. Apesar do teto final idêntico, as curvas de convergência (gerações 25, 50, 100 e 200, disponíveis no JSON exportado) ainda diferem entre as taxas de mutação antes de alcançá-lo. Com população 60 e 8 neurônios ocultos, a mutação 0,16 lidera no início (59,13 na geração 25, contra 57,48 das outras duas) mas estagna logo depois (62,43 na geração 200); a mutação 0,08 é a que mais se aproxima do teto antes do fim (68,31 já na geração 200); e a mutação 0,04 fica no meio (65,68 na geração 200). O padrão é o mesmo do experimento anterior – mutação alta explora rápido e depois trava, mutação baixa refina mais lentamente –, só que agora as três acabam convergindo ao mesmo teto por volta da geração 222, em vez de pararem em valores finais diferentes. O compromisso clássico entre exploração e explotação continua presente na trajetória; deixou de ser visível no resultado final porque o resultado final deixou de ter margem para diferenciá-las.

Custo computacional: a população continua sendo o único fator relevante. A última coluna da Tabela 3 confirma, com o ambiente novo, a mesma conclusão do experimento anterior: o tempo de treinamento escala com o tamanho da população (3,3 a 3,6 s em população 30; 6,3 a 8,2 s em população 60; 11,3 a 20,5 s em população 120 – um fator de aproximadamente 3× a 6× entre os extremos) e varia muito menos, e sem uma direção consistente, com o número de neurônios ocultos. A razão é a mesma de antes: o custo é dominado pela quantidade de episódios simulados por geração (um por indivíduo da população), não pelo tamanho da rede avaliada em cada episódio – mesmo com 7 entradas em vez de 6, a rede continua pequena demais para que sua propagação pese diante do custo de simular o ambiente. A maior variação não-monotônica entre arquiteturas nesta rodada (por exemplo, 120/16 levando 20,5 s contra 11,3 s de 120/4) é compatível com a nova mecânica de fim antecipado de episódio por invasão de torre:

episódios que terminam mais cedo custam menos tempo de simulação, e qual genoma de uma população específica sofre invasão antecipada em qual geração é, em si, estocástico – uma fonte de ruído de tempo que o ambiente anterior, sem fim antecipado, não tinha.

Conclusão e escolha dos hiperparâmetros. A conclusão para o RadarTorres permanece de custo-benefício, agora ainda mais categórica: com todas as onze configurações atingindo o mesmo teto de fitness na mesma geração, não há absolutamente nenhuma evidência – nem mesmo a variação de 3% observada antes do endurecimento do ambiente – que justifique o gasto adicional de uma população maior ou de uma camada oculta maior. A configuração de produção (população 60, 8 neurônios ocultos, mutação 0,08) permanece sustentada não por superar as demais, mas por ser a escolha de fábrica mais testada e por não haver custo de oportunidade mensurável em mantê-la.

Duas ressalvas, na mesma linha das já apontadas na Seção 2.2, delimitam o alcance desta conclusão. A primeira é que a saturação observada é uma propriedade do conjunto FIXO de cenários táticos usado nesta bateria (`CatalogoCenarios.TodosComScript`), e não necessariamente do treinamento de produção, que também avalia cenários aleatórios – o teto idêntico documentado aqui não implica que o controlador de produção (Seção 2.5) tenha convergido do mesmo jeito sob nascimentos sorteados e um horizonte de treinamento muito maior. A segunda é a mesma de antes: cada configuração corresponde a uma única execução (não a uma média de sementes independentes), de modo que a coincidência de picos exatamente na geração 222 em onze execuções sem semente fixa compartilhada é, ela própria, um indício de quão determinístico o teto se tornou nesta bateria específica – mas não prova que o mesmo valha fora dela. Repetir esta mesma bateria com cenários aleatórios, em vez dos fixos, é o próximo passo natural para saber se a saturação observada é exclusiva dos cenários táticos obrigatórios ou uma característica mais geral do ambiente endurecido.

Teste direto: o teto é da rede ou da duração fixa do episódio? Uma forma mais direta de responder a essa pergunta, sem depender dos cenários táticos fixos, é remover a própria causa suspeita: a duração fixa do episódio. Foi adicionado um modo de avaliação (`ParametrosTreinamento.SemLimiteDeDuracao`) em que o episódio IGNORA os 45 s normais e só termina quando um alvo de fato rompe uma torre (ou aos 1800 s de uma salvaguarda, nunca atingida nos testes) – a escalada de dificuldade continua usando os 45 s como referência de ritmo, só o corte abrupto por tempo é desligado. O genoma de produção já treinado (pico de fitness 193,57 em treinamento, dentro do limite normal de 45 s) foi avaliado assim em 40 episódios independentes, sem nenhuma mudança na rede. O resultado desfaz a pergunta original em duas partes, não uma resposta única: o fitness MÁXIMO observado sobe para 443,23 – quase o triplo do teto de produção e acima da meta de 300 que motivou este experimento –, confirmando que a duração fixa de fato interrompia episódios que ainda tinham pontuação disponível; mas o fitness MÉDIO (82,35) e a duração média até a invasão de uma torre (29,2 s, variando de 5,4 s a 87,4 s) mostram que boa parte dos episódios termina por falha real do controlador bem ANTES dos 45 s originais, não por causa do corte. Em outras palavras, o teto observado em produção não é puramente "o episódio é curto demais" – é a combinação de um corte de tempo que, de fato, tira pontuação de episódios bem-sucedidos, com uma robustez do controlador ainda insuficiente para evitar falhas precoces numa fração

considerável dos cenários. Isso aponta para dois trabalhos futuros distintos, não um só: alongar o horizonte de treinamento (para a rede aprender a explorar episódios longos e bem-sucedidos) e reduzir a variância entre *seeds* (o número inteiro que inicializa o gerador de números pseudoaleatórios do episódio – a mesma seed sempre produz exatamente os mesmos alvos, nas mesmas posições e velocidades, byte a byte) (para a rede falhar com menos frequência nos cenários desfavoráveis), sendo este segundo, pelos dados aqui, o fator que mais pesa na média atual.

A variância é da rede, do ambiente, ou da avaliação do algoritmo genético? O teste anterior levanta uma suspeita incômoda: se o teto sobe para 443 mas a média continua baixa, o sistema está aprendendo algo consistente ou só tirando sorte? Três medições, nesta ordem, isolam a resposta. Primeiro, o desvio padrão de fitness do mesmo genoma de produção foi medido dentro do regime NORMAL de 45 s (não o modo sem limite), em 50 episódios independentes: desvio de 63,65 contra uma média de 66,69 – ou seja, a variância já é da mesma ordem de grandeza da própria média mesmo sem remover o corte de tempo, o que descarta a hipótese de que ela fosse um artefato só do modo sem limite. Segundo, o MESMO genoma foi avaliado em modo `HeuristicaPura` (rede completamente ignorada, ajuste zero, dispara sempre que a heurística permite) nas MESMAS 50 seeds: desvio de 67,36 e média de **74,34** – estatisticamente indistinguível da rede treinada, e com média até superior. Isso aponta a variância para o ambiente (que alvos nascem, em que quadrante, em que instante), não para a política de decisão – mesmo uma heurística determinística sofre a mesma montanha-russa de sorte, porque um único alvo nascendo no lugar errado no início do episódio já basta para invadir uma torre em poucos segundos, antes de qualquer controlador conseguir reagir.

A comparação acima, porém, revelou um problema diferente e mais preocupante: a rede treinada estava perdendo, em média, para a heurística que ela deveria apenas refinar. A hipótese seguinte foi que `EpisodiosPorAvaliacao` (quantos episódios são sorteados e tirada a média para avaliar CADA genoma a cada geração) estivesse baixo demais – 3, antes deste teste – para a seleção do algoritmo genético distinguir genoma melhor de genoma apenas sortudo, dado que o próprio desvio por episódio já é maior que a diferença de qualidade que se quer medir. O valor foi elevado para 12 e um novo genoma de produção foi treinado do zero com o restante dos parâmetros idêntico (1000 gerações, população 60, 8 neurônios ocultos, mutação 0,08, cenários aleatórios). O resultado, infelizmente, não confirma essa hipótese: nas mesmas 50 seeds, a rede treinada com 12 episódios por avaliação obteve média 65,02 (desvio 60,57) contra os mesmos 74,34 (desvio 67,36) da heurística pura – o GANHO esperado não apareceu, e o desvio continua do mesmo tamanho. Elevar o número de episódios por avaliação reduz o ruído da MEDIÇÃO de fitness que o algoritmo genético usa para selecionar, mas não resolve se o problema estiver em outro lugar – e os dados aqui sugerem que está: como a seleção de torre já é dominada pelo termo heurístico na pontuação combinada (Seção 2.3.4), é a decisão de DISPARO (`confiancaDisparo > 0`, inteiramente da rede em modo híbrido, sem veto heurístico de oportunidade) a candidata mais provável a estar aprendendo a hesitar ou recusar disparos que a heurística pura, sem essa hesitação, aproveitaria.

Para separar "este genoma específico saiu ruim" de "o método de treino tende a produzir isto", o treinamento (1000 gerações, 12 episódios por avaliação, restante idêntico) foi repetido com mais três sementes do algoritmo genético (2, 3 e 4), cada uma avaliada

nas mesmas 50 seeds de teste contra o mesmo controle de heurística pura (74,34 de fitness médio, por não depender de genoma nenhum). O resultado é sistemático, não uma execução sortuda ruim: as QUATRO sementes ficaram abaixo da heurística pura – 65,02 (semente 1), 71,56 (semente 2), 65,09 (semente 3) e 64,18 (semente 4) –, numa margem pequena mas consistentemente negativa em todas elas. Isso descarta a hipótese de infortúnio de uma única execução do algoritmo genético e aponta para algo estrutural no método de treino: dado que a seleção de torre já é dominada pelo termo heurístico, é bastante provável que seja de fato a decisão de disparo da rede – isolada, sem veto heurístico de oportunidade – quem aprende um comportamento levemente pior do que simplesmente disparar sempre que permitido.

Confirmação direta: a rede hesita no gatilho. A hipótese anterior foi medida diretamente, em vez de apenas inferida por eliminação. Toda vez que os vetos inegociáveis (zona morta, certeza mínima, tempo mínimo de mira) já liberam o disparo – exatamente o ponto em que a heurística pura dispararia sempre, sem exceção – o ambiente agora conta essa "oportunidade de disparo" e se a rede, em modo híbrido, a recusou (`EstadoEpisodio.OportunidadesDeDisparo/DisparosRecusadosPelaRede`, somados ao longo de 50 episódios por genoma). O resultado confirma a hipótese sem ambiguidade: as CINCO redes treinadas neste trabalho (a de produção original e as quatro sementes da seção anterior) recusam disparo numa fração nada desprezível das oportunidades legítimas – 10,0%, 34,3%, 10,1%, 55,9% e 51,7%, respectivamente –, contra 0,0% da heurística pura (que, por definição, nunca recusa). Quanto maior essa taxa de recusa, pior tende a ser o desempenho relativo à heurística pura: as sementes 3 e 4 (55,9% e 51,7% de recusa) são também as que ficam mais distantes do controle na comparação de fitness da seção anterior. O mecanismo suspeitado está, portanto, confirmado: a rede não está aprendendo a escolher melhor QUAL torre age (papel já dominado pela heurística na pontuação combinada), mas sim um critério de confiança de disparo pior do que simplesmente "disparar sempre que autorizado" – um resultado negativo honesto, mas instrutivo, sobre os limites desta formulação específica do controlador híbrido e um ponto de partida concreto para revisar a função de fitness (por exemplo, penalizando explicitamente a recusa de uma oportunidade legítima, não só recompensando o abate) como trabalho futuro imediato.

Removendo de vez a duração fixa: o teto é, em parte, do ambiente – mas não só dele. Como teste final desta investigação, a duração de 45 s do episódio foi removida também do TREINAMENTO em si (não só da avaliação pós-treino da Seção anterior): um novo genoma de produção foi treinado do zero, 1000 gerações, com `SemLimiteDeDuracao` ativo durante toda a evolução – o episódio só termina quando um alvo de fato rompe uma torre. O pico de fitness alcançado em treinamento subiu para 211,98 (geração 735), contra os 193,57 do treinamento original limitado a 45 s, e a avaliação fora de amostra (50 seeds novas, também sem limite de duração) chegou a um fitness MÁXIMO de 245,94 para a rede e 434,72 para a heurística pura – muito acima de qualquer teto visto anteriormente. Isso confirma, de forma definitiva, que parte real do teto de produção vinha da duração fixa do episódio: sem ela, o sistema – rede ou heurística – consegue, em episódios favoráveis, pontuações 2 a 3 vezes maiores.

Removida essa causa, porém, o segundo problema PERMANECE do mesmo tamanho:

o fitness MÉDIO da rede treinada sem limite de duração (69,28) continua abaixo da heurística pura no mesmo regime (89,69), e a taxa de recusa de disparo desse genoma é 35,8% – dentro da mesma faixa (10% a 56%) medida nas cinco redes treinadas com episódio limitado. Em outras palavras, dar mais tempo de episódio não ensinou a rede a disparar melhor; só deu a ela (e à heurística, igualmente) mais tempo para acumular pontos quando a sorte do episódio é favorável. Os dois problemas identificados nesta investigação são, portanto, independentes: a duração fixa do episódio limitava o TETO alcançável (resolvido removendo o limite), e a hesitação aprendida na decisão de disparo limita a MÉDIA alcançável – este segundo problema aponta para a função de fitness, não para o ambiente, e foi atacado diretamente a seguir.

Corrigindo a função de fitness: penalizar a recusa em si. Como a função de fitness, até este ponto, só recompensava abate e penalizava fuga/invasão – nunca custando nada à rede simplesmente NÃO disparar numa oportunidade legítima –, foi adicionado um termo explícito para isso: `ParametrosTreinamento.PenalidadeRecusaDisparo` subtrai 3,0 pontos de fitness toda vez que `EstadoEpisodio.DisparosRecusadosPelaRede` é incrementado (ver parágrafo anterior), em tempo real, durante o próprio episódio de treino – não é uma penalidade pós-hoc, aplicada só na análise. Um genoma de produção foi retreinado do zero com esse termo ativo, mesmo ambiente sem limite de duração, mesmas 1000 gerações. O resultado confirma que a hesitação era, de fato, um problema da função de fitness, corrigível por ela mesma: a taxa de recusa de disparo caiu de 35,8% para **0,0%** – a rede evoluída passou a se comportar, no gatilho, EXATAMENTE como a heurística pura (nunca recusa). O fitness médio, avaliado nas mesmas 50 seeds de teste, subiu de 69,28 para 86,09 – a 4% da heurística pura (89,69), contra os quase 30% de distância antes da correção – e o fitness máximo (434,59) ficou praticamente empatado com o teto da heurística pura (434,72). A rede, porém, não passou a SUPERAR a heurística pura: convergiu para replicá-la na decisão de disparo, sem demonstrar ganho próprio nessa decisão específica – o ganho de qualidade observado nesta seção vem inteiramente de eliminar um defeito (a hesitação), não de a rede ter aprendido algo que a heurística não sabia. Ainda assim, é o resultado mais consistente da neuroevolução deste trabalho: confirma que o fitness, quando mede exatamente o comportamento que se quer corrigir, é suficiente para corrigi-lo, sem precisar de nenhuma mudança na arquitetura da rede ou no ambiente de treino.

Terceira comparação: a mesma rede, sem a camada heurística, supera as outras duas. A ablação heurística-*vs*-rede já conduzida neste trabalho (Seção 2.3.6, modo `RedePura`) havia sido medida sob a arquitetura e o ambiente antigos, num genoma treinado à parte para esse modo – não a MESMA rede de produção, só sem a heurística. Para completar o quadro desta investigação, a rede de produção corrigida (penalidade de recusa, treinada sem limite de duração) foi avaliada também em `RedePura` – mesmo genoma, mesmas 50 seeds de teste, mas sem a heurística filtrar candidatas por quadrante nem impor o tempo mínimo de mira de 0,3 s. O resultado é o melhor dos três regimes medidos nesta seção:

A MESMA rede, livre do filtro heurístico de candidatas e do tempo mínimo de mira obrigatório, supera tanto o modo híbrido quanto a heurística pura – 114,12 contra 89,69 e 86,09, respectivamente. A explicação mais provável está no próprio tempo mínimo de

Table 4: Mesma rede de produção (treinada com a penalidade de recusa), avaliada nos três modos de decisão, 50 seeds de teste, sem limite de duração de episódio.

Modo	**Fitness médio**	**Fitness máximo**	**Taxa de recusa**
Híbrido (produção)	86,09	434,59	0,0%
Heurística pura	89,69	434,72	0,0%
Rede pura	**114,12**	**449,00**	0,2%

mira: 0,3 s de espera obrigatória antes de qualquer disparo, nesta versão endurecida do ambiente (onde um único alvo não interceptado a tempo pode invadir uma torre e encerrar o episódio), é tempo que o modo rede pura não perde – ela já pode disparar no instante em que a certeza mínima é atingida. Esse resultado não implica que a heurística deva ser removida de produção: as regras heurísticas (Seção 2.3.5) existem por segurança e previsibilidade operacional, não só por desempenho de fitness, e o tempo mínimo de mira em particular evita disparos por reflexo em identificações ainda instáveis. O achado, em vez disso, é um indício concreto de que o CUSTO de segurança desse veto específico é medível e relativamente alto neste ambiente – 24 pontos de fitness médio, cerca de 28% – e fica registrado como candidato a revisão: talvez o tempo mínimo de mira devesse ser menor, ou condicionado à urgência do alvo, em vez de um valor fixo único para qualquer situação.

2.4 Diagrama de pacotes

A organização interna da aplicação desktop foi documentada por meio de um diagrama de pacotes, diagrama estrutural da UML que representa a divisão lógica do sistema em pacotes e as dependências existentes entre eles, sem detalhar atributos ou métodos das classes neles contidas. No RadarTorres, cada pacote corresponde a um *namespace* do projeto `RadarTorres.App`, coincidente com uma pasta do código-fonte, e o diagrama foi obtido diretamente do código da *branch* principal do repositório: cada seta representa ao menos uma diretiva `using RadarTorres.App.*` (ou uma referência `clr-namespace` em XAML) presente nos arquivos do pacote de origem, de modo que o diagrama descreve a estrutura efetivamente implementada, e não apenas a pretendida.

O fluxo principal de dependências segue a organização do padrão MVVM, de cima para baixo: Views → ViewModels → Services → Repositories → Data → Models. O pacote Models ocupa a posição mais estável do sistema – todos os demais dependem dele e ele não depende de nenhum outro –, o que é desejável para as entidades de domínio, e a separação entre as interfaces de Repositories e a implementação em CSV do pacote Data é o que viabiliza a futura migração para um banco relacional sem alterar ViewModels ou Services. A construção do diagrama a partir do código revelou, também, dependências que formam ciclos entre pacotes: o `NavigationService` importa *namespaces* de Views para instanciar telas (Services → Views, fechando o ciclo com Views → ViewModels → Services), e o `DataSeeder`, responsável pela carga inicial de dados, importa Models, Repositories e Services (Data → Repositories/Services, fechando o ciclo com Repositories/Services → Data). Confirmados em `NavigationService.cs` e `DataSeeder.cs` no código atual, esses dois ciclos continuam presentes, mas a figura acima, regenerada ao nível de pacote e não de classe, não os destaca visualmente – ficam registrados apenas neste parágrafo, em vez de perdidos. Eles não impedem o funcionamento da aplicação, pois todos os pacotes

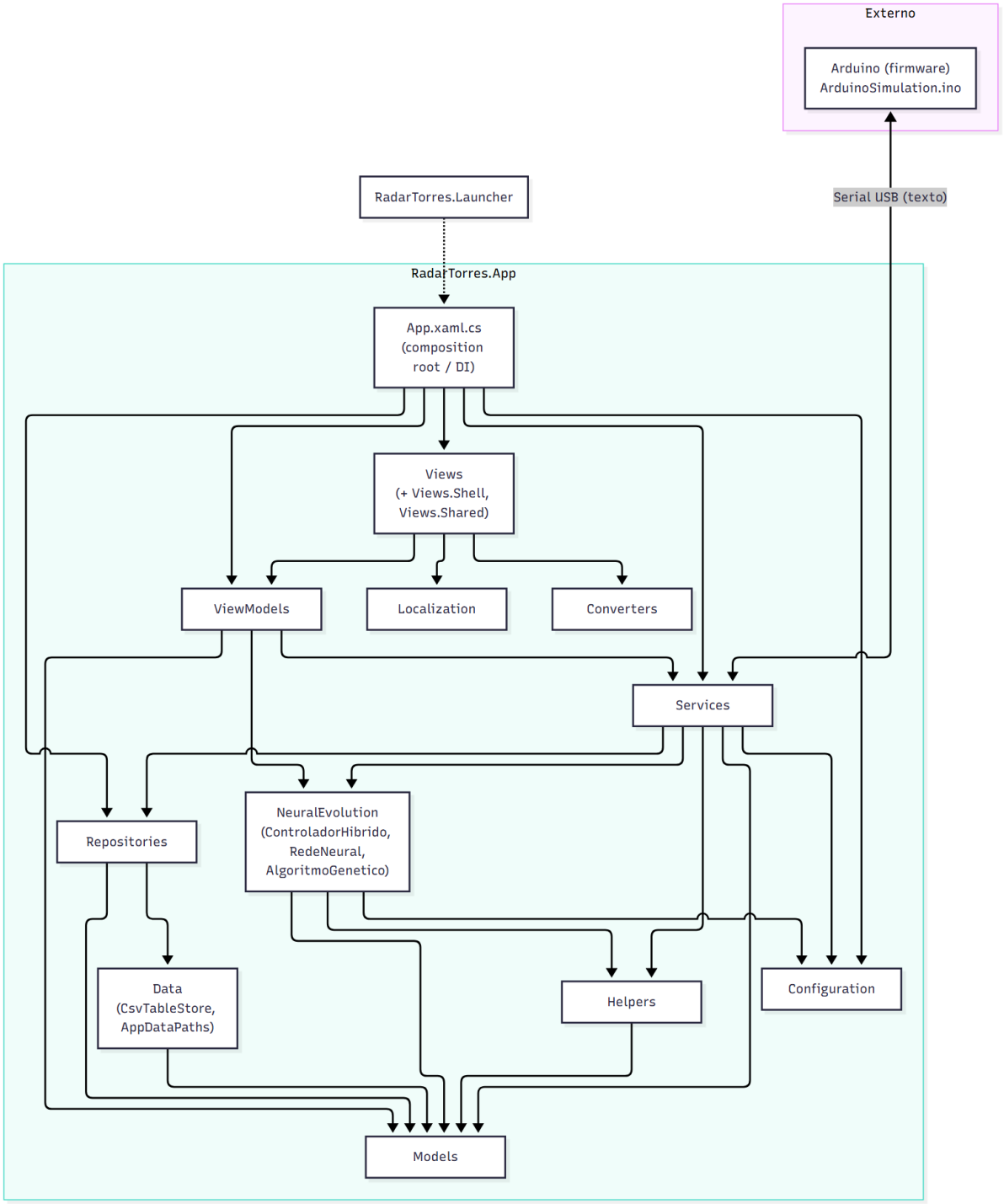


Figure 3: Diagrama de pacotes do RadarTorres. Setas em vermelho indicam dependências que formam ciclos (dos autores).

pertencem ao mesmo *assembly*, mas aumentam o acoplamento e dificultam testes isolados e futuras separações em projetos distintos.

2.5 Modelo de dados

O protótipo não depende de um servidor de banco de dados externo: a persistência usa **SQLite** (`Microsoft.Data.Sqlite`, arquivo único `radartorres.db` na pasta de dados do usuário), com `PRAGMA foreign_keys = ON` garantindo chaves estrangeiras reais – estágio mais avançado do que o modelo em CSV descrito em versões anteriores deste documento (ver nota histórica adiante). O modelo de dados abaixo foi levantado diretamente do código-fonte já implementado, não de um diagrama projetado antes da codificação. O schema atual tem onze tabelas, listadas na Tabela 5: sete cadastrais/de auditoria, já cobertas pelo levantamento em `conteudo_modelo_banco_dados.tex` (pasta `Banco_de_Dados` dos documentos entregáveis), e quatro exclusivas do treinamento neuroevolutivo, que persistem a evolução do controlador híbrido (Seção 2.3.4) entre execuções.

Table 5: Tabelas do schema SQLite atual (`schema.sql` e os `Sqlite*Repository.cs`).

Tabela	**Finalidade**
`usuarios`	Contas de acesso, perfil (Administrador/Operador/Visualizador) e autenticação
`objetos_detectados`	Histórico de ocorrências de detecção (radar e câmera, por quadrante)
`ocorrencia_estados`	Auditoria das transições de estado de cada ocorrência (só inserção)
`cameras`	Câmera responsável por cada quadrante (Q1–Q4)
`acoes_realizadas`	Auditoria de acionamentos efetivamente executados (só inserção)
`modo_atual_torre`	Auditoria de troca de modo de operação (só inserção)
`preferencias_usuario`	Preferências de interface por usuário (tema, idioma, sidebar)
`classificacoes_alvo`	Classificação de perigo do alvo por velocidade estimada, vinculada a `objetos_detectados` por FK
`historico_treinamento_geracoes`	Fitness (melhor e médio) e contagem de alvos abatidos/escapados a cada geração do algoritmo genético
`marcos_fitness_treinamento`	Agregado de fitness por bloco de gerações, para acompanhar a convergência do treino sem gravar geração a geração
`instantaneos_comportamento_melhor_gene`	Snapshot do comportamento do melhor indivíduo em marcos do treino: fitness, taxa de acerto, taxa de cobertura e o próprio genoma (JSON)

O modelo de dados atual (SQLite, 11 tabelas, chaves estrangeiras reais) é apresentado a seguir em três grupos, por legibilidade: o núcleo de usuários e preferências (Figura 4), o núcleo de detecção, câmeras e auditoria (Figura 5) e as tabelas de treinamento neuroevolutivo, independentes entre si (Figura 6) – diagramas entidade-relacionamento levantados a partir do código-fonte do projeto.

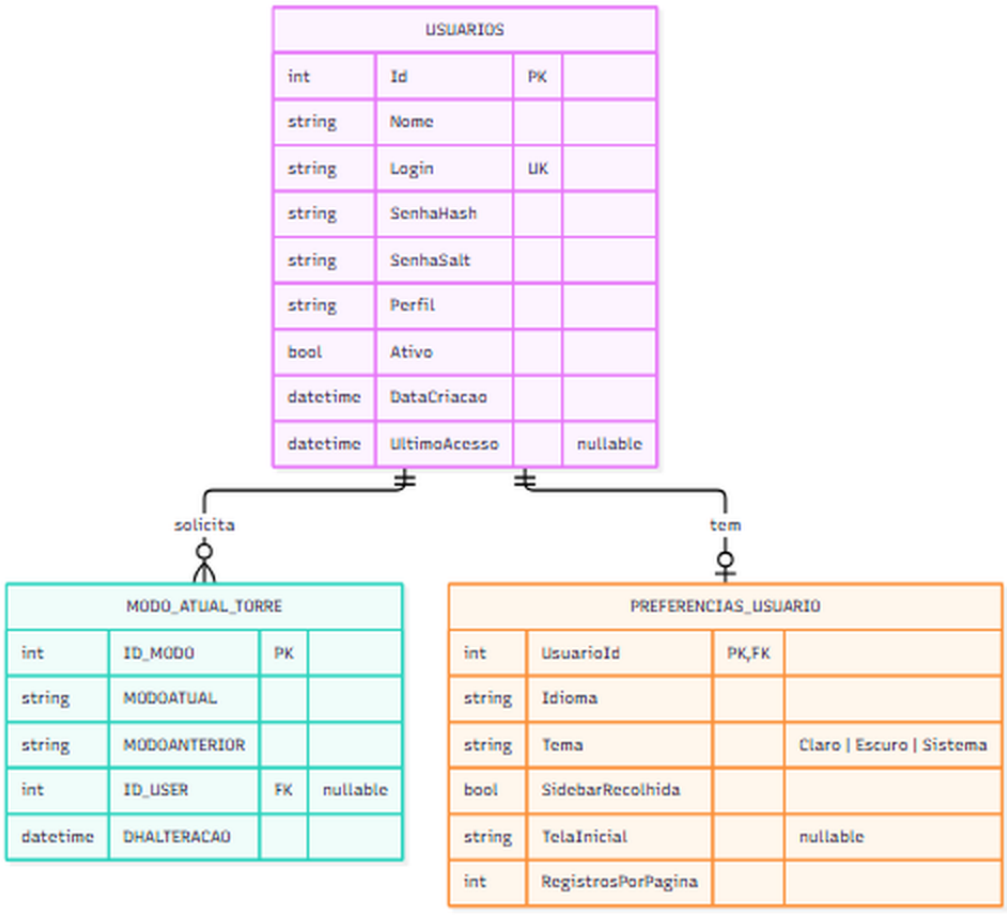


Figure 4: Núcleo de usuários: `usuarios`, `modo_atual_torre` e `preferencias_usuario` (dos autores).

`usuarios` se relaciona 1:1 com `preferencias_usuario` (chave `UsuarioId` → `usuarios.Id`) e 1:N com `modo_atual_torre` (`ID_USER` → `usuarios.Id`) – em ambos os casos por chave estrangeira real de inteiro, não mais por login em texto. `objetos_detectados` se relaciona 1:N com `acoes_realizadas` (quando há alvo associado ao acionamento), com `ocorrencia_estados` (histórico de transição de estado) e com `classificacoes_alvo` (`ObjetoDetectadoId` → `objetos_detectados.Id`); `cameras` se relaciona 1:N com `objetos_detectados` pelo quadrante monitorado. As quatro tabelas de treinamento não se relacionam por FK com as demais nem entre si: são séries temporais independentes, cada uma identificada pelo número da geração (`GERACAO`/`GeracaoMarco`) e pelo instante de registro, cobrindo da granularidade mais fina (`historico_treinamento_geracoes`, uma linha por geração) à mais agregada (`instantaneos_comportamento_melhor_gene`, uma linha por marco, com o genoma completo em JSON).

Como registro histórico do projeto, o modelo original em arquivos CSV tinha seis entidades (sem `ChamadoAjuda`, removida do código) e nenhuma chave estrangeira real – os vínculos

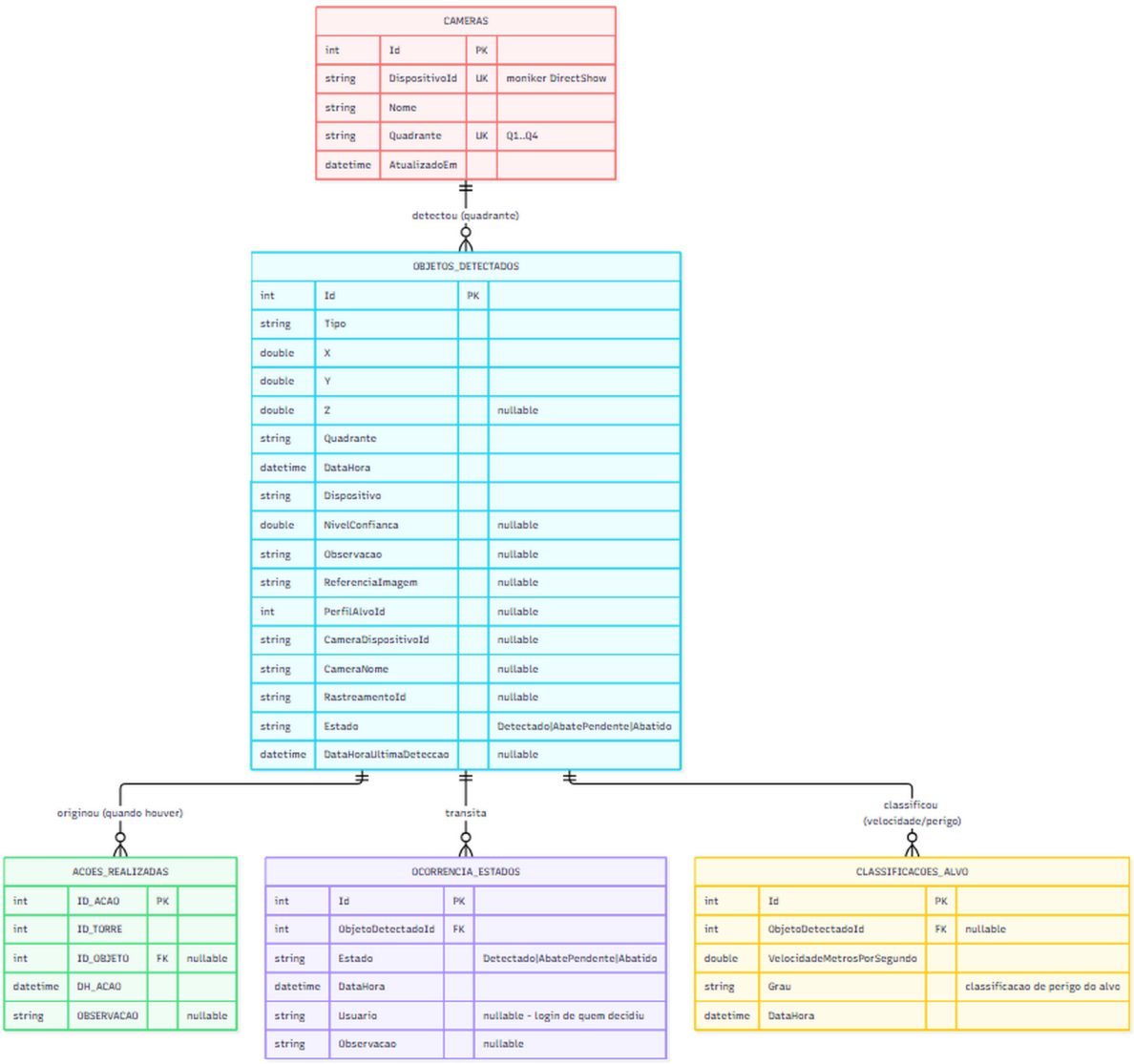


Figure 5: Núcleo de detecção e auditoria: `cameras`, `objetos_detectados`, `acoes_realizadas`, `ocorrencia_estados` e `classificacoes_alvo` (dos autores).

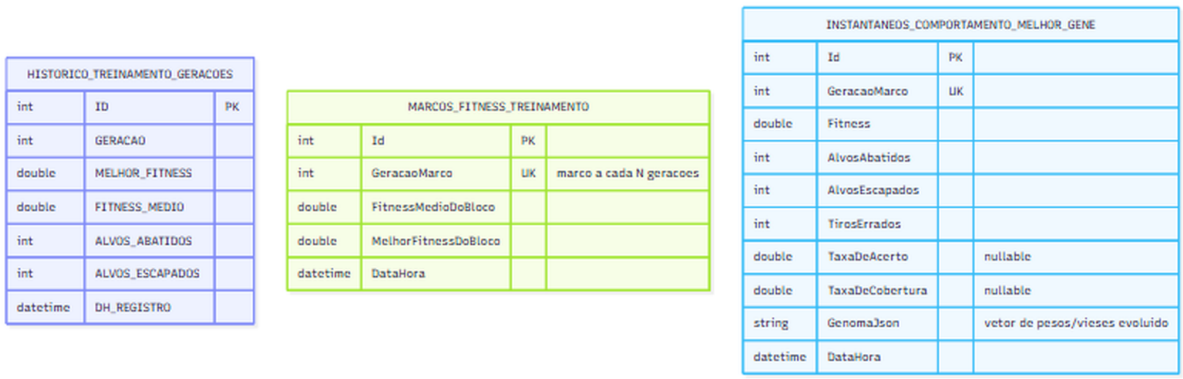


Figure 6: Tabelas de treinamento neuroevolutivo – séries temporais independentes entre si: `historico_treinamento_geracoes`, `marcos_fitness_treinamento` e `instantaneos_comportamento_melhor_gene` (dos autores).

entre `Usuario` e `AcaoRealizada/ModoAtualTorre` eram feitos por login em texto. A migração efetivamente realizada seguiu a proposta elaborada a partir desse modelo original para as sete entidades que já existiam, mas foi além dela ao acrescentar as quatro tabelas de treinamento neuroevolutivo, inexistentes na época em que a proposta foi desenhada.

3 Conclusão

Este trabalho descreveu o RadarTorres como um protótipo de rastreamento e seleção automática de alvos construído sobre sensoriamento ultrassônico, visão computacional e um controlador híbrido – heurística determinística que filtra e veta, rede neural pequena que decide dentro do espaço já validado, treinada por algoritmo genético. A contribuição mais original deste trabalho é empírica: a identificação e correção, documentada por commit, de uma falha em que a certeza de identificação do alvo era sorteada uma única vez e nunca atualizada, mantendo cerca de 70% dos alvos simulados permanentemente abaixo do limiar de disparo e mascarando a estagnação evolutiva sob a aparência de um treino funcional. A Tabela 1 (Seção anterior) quantifica o efeito dessa correção: o fitness médio da população, estagnado em torno de 9 ao longo de 25.500 gerações antes da correção, passou a acompanhar de perto o fitness do melhor indivíduo depois dela, e a cobertura de alvos disparáveis foi de uma fração minoritária para 100% nas amostras pós-correção.

As limitações do protótipo, discutidas ao longo do texto, concentram-se em três pontos: (i) o hardware físico ainda não foi montado – toda validação é feita em simulação, com modelos (certeza de identificação, probabilidade de erro de disparo) assumidos e não calibrados com dados de campo; (ii) a comunicação entre a visão computacional e o Arduino para guiar o apontamento de pan/tilt foi projetada mas não implementada, permanecendo como integração pendente; e (iii) a propriedade de projeto que limita o controlador híbrido, no pior caso, ao comportamento puramente heurístico (nunca a uma decisão arbitrária da rede) ainda não foi submetida a validação em hardware físico real.

Os trabalhos futuros identificados neste artigo são, portanto: montagem e validação do hardware físico das cinco torres; implementação do comando de apontamento no protocolo serial e da fusão entre a posição estimada pela visão e a distância ultrassônica; calibração dos modelos de certeza e de erro de disparo com dados de campo, substituindo as premissas simuladas; extensão da seleção de torre para a abordagem multiameaça da literatura de TEWA, hoje deixada de lado em favor da abordagem por proximidade; e, caso a capacidade da rede se mostre o fator limitante do controlador em vez dos hiperparâmetros da busca evolutiva, avaliação de uma arquitetura maior à luz dos resultados de Such et al. (2017).

Referências

ALSHIEKH, M. et al. *Safe Reinforcement Learning via Shielding*. 2017.

BAEK, I. et al. *CurbScan: Curb Detection and Tracking Using Multi-Sensor Fusion*. 2020.

HUANG, S.; ONTAÑÓN, S. *A Closer Look at Invalid Action Masking in Policy Gradient Algorithms*. 2020.

MAJONI, L. et al. Camera and ultrasonic sensor fusion for electronic travel aid: design and performance analysis. *MATEC Web of Conferences*, v. 417, p. 10005, 2025.

NAEEM, H. et al. *A Novel Two-Stage Dynamic Decision Support based Optimal Threat Evaluation and Defensive Resource Scheduling Algorithm for Multi Air-borne threats*. 2009.

NAEEM, H. et al. *A Novel Two-Staged Decision Support based Threat Evaluation and Weapon Assignment Algorithm, Asset-based Dynamic Weapon Scheduling using Artificial Intelligence Techniques*. 2009.

SUCH, F. P. et al. *Deep Neuroevolution: Genetic Algorithms Are a Competitive Alternative for Training Deep Neural Networks for Reinforcement Learning*. 2017.